Tool-Augmented Conversation Simulator (TACS)

Complete Technical Documentation

Version: 0.1.0 Repository: tool-augmented-conversation-simulator Language: Python 3.11+

Table of Contents

1. Executive Summary
2. Why This Project Exists
3. Repository Layout
4. End-to-End Dataflow
5. Configuration System
6. LLM Abstraction
7. CLI Commands
8. Registry Subsystem
9. Graph Subsystem
10. Agent Subsystem
11. Execution Subsystem
12. Memory Subsystem
13. Data Contracts and JSON Output
14. Testing Strategy
15. Determinism and Reproducibility
16. Error Handling Philosophy
17. Design Decisions and Rationale
18. Known Constraints and Trade-offs
19. Extension Points
20. Packaging and Dependencies
21. Operational Runbook
22. Security and Data Considerations
23. File-by-File Index
24. Quick Reference Commands

---

1. Executive Summary

TACS is an offline synthetic data generation system that creates multi-turn, multi-tool conversational traces for training and evaluating tool-using AI assistants.

It ingests ToolBench API definitions, constructs a typed tool registry, derives a compatibility graph between endpoints, samples realistic tool-call chains, simulates multi-agent conversations around those chains, executes deterministic mock tool calls, and writes JSONL datasets with metadata suitable for quality checks and comparative experiments.

Core outcomes:

- Reproducible synthetic conversation generation using seeded randomness.
- Structured tool usage traces with tool calls and tool outputs.
- Session and corpus memory integration through mem0.
- Rule-based validation against hard quality constraints.
- CLI-based workflow for build, generate, validate, and metrics.

---

2. Why This Project Exists

The Problem

Training and evaluating tool-using AI assistants requires large datasets of conversations where an AI correctly chains together multiple API calls. Collecting such data from real users is expensive, slow, and raises privacy concerns. Real data is also impossible to label with ground-truth tool arguments or orchestrate deliberately to cover specific tool-chain patterns.

What TACS Provides That Real Data Cannot

Property	Real Data	TACS Synthetic Data
Scale	Limited by user traffic	Unlimited — generate as many as needed
Ground truth arguments	Unavailable	Always present (MockExecutor echoes inputs)
Coverage of rare tool combos	Random, uncontrolled	Systematic via graph sampling
Privacy concerns	Yes	None — fully synthetic
Reproducibility	None	Exact — seeded RNG
Cost	High (labelling, API calls)	Low — offline after build phase

The Experiment Design

TACS is specifically designed to run a **controlled A/B experiment** comparing two generation conditions:

- **Run A** (`--no-corpus-memory`): Each conversation is planned from scratch. No prior conversations influence scenario planning.
- **Run B** (default, `--corpus-memory`): The planner reads summaries of prior conversations from a corpus-scoped vector store before generating a scenario. This encourages diversity and thematic coherence across the dataset.

The `tacs metrics --input-a --input-b` command then computes delta metrics to determine whether corpus memory improves dataset quality.

---

3. Repository Layout

```
tool-augmented-conversation-simulator/
├── tacs/                          # Main package
│   ├── config.py                  # Pydantic settings model + global singleton
│   ├── llm.py                     # Backend-agnostic LLM client wrapper
│   ├── cli.py                     # Click CLI: build / generate / validate / metrics
│   ├── registry/
│   │   ├── loader.py              # Loads ToolBench JSONs, normalizes to typed models
│   │   ├── registry.py            # In-memory tool registry with O(1) lookups
│   │   └── models.py              # Pydantic models: Tool, Endpoint, Parameter
│   ├── graph/
│   │   ├── builder.py             # Builds NetworkX DiGraph + COMPATIBLE_WITH edges
│   │   ├── models.py              # NodeType, EdgeType enums + node_id() helper
│   │   └── sampler.py             # ToolChainSampler: multi_step / parallel / hybrid
│   ├── agents/
│   │   ├── base.py                # BaseAgent abstract class
│   │   ├── models.py              # Message, ToolCall, ToolOutput, Conversation, etc.
│   │   ├── sampler_agent.py       # Wraps ToolChainSampler with fallback
│   │   ├── planner_agent.py       # Plans scenario, reads corpus memory
│   │   ├── user_proxy.py          # Generates user utterances
│   │   ├── assistant_agent.py     # Decides clarify/tool_call/respond + fills args
│   │   ├── validator_agent.py     # Rule-based validation of completed conversations
│   │   └── pipeline.py            # Orchestrates all agents; writes JSONL output
```

```
│   ├── execution/
│   │   └── executor.py            # MockExecutor: deterministic mock tool responses
│   └── memory/
│       └── store.py               # MemoryStore: mem0 wrapper with session/corpus scopes
├── tests/
│   ├── unit/
│   │   ├── test_registry.py
│   │   ├── test_graph.py
│   │   └── test_memory.py
│   └── e2e/
│       └── test_pipeline.py
├── data/                          # ToolBench JSON files (user-provided)
├── artifacts/                     # Build outputs: registry.pkl, graph.pkl, build_meta.json
├── output/                        # Generated JSONL: run_a.jsonl, run_b.jsonl
├── docs/                          # This file
├── README.md
├── DESIGN.md
├── pyproject.toml
└── requirements.txt
```

---

4. End-to-End Dataflow

4.1 Build Phase (`tacs build`)

```
1. Load ToolBench JSON files from data/toolenv/tools/<category>/*.json
2. Normalize raw records → Tool, Endpoint, Parameter models
3. Save ToolRegistry as artifacts/registry.pkl
4. Build NetworkX DiGraph:
   - Add TOOL nodes
   - Add CONCEPT nodes (categories); connect via TAGGED_WITH
   - Add ENDPOINT nodes; connect Tool→Endpoint via HAS_ENDPOINT
   - Add PARAMETER nodes; connect Endpoint→Parameter via HAS_PARAMETER
   - Add RESPONSE_FIELD nodes; connect Endpoint→ResponseField via RETURNS
   - Add COMPATIBLE_WITH edges (response field name matches required param name)
5. Save graph as artifacts/graph.pkl
6. Write artifacts/build_meta.json with counts and timestamp
```

4.2 Generation Phase (`tacs generate`) — Per Conversation

```
SamplerAgent
└── ToolChainSampler samples a chain (multi_step / parallel / hybrid)
    └── Enforces ≥3 steps, ≥2 distinct tools

PlannerAgent
├── If corpus_memory_enabled: search corpus for top-5 prior summaries
├── LLM generates {scenario, domain, pattern_type} as JSON
└── Fallback: derive from tool names + domain keyword map

Conversation Loop (max_turns=10):
│
├── UserProxyAgent
│   ├── Turn 1: LLM generates opening message from scenario
│   │   (intentionally vague to encourage clarification)
│   └── Later turns: LLM responds to assistant clarification
│
```

```
├── AssistantAgent (rule-based decision)
│   ├── If final=True → LLM writes closing response (action=respond)
│   ├── Read session memory (non-first steps only)
│   ├── Rule: if step==0 AND missing required params AND no prior assistant msg
│   │         → action=clarify (LLM writes the question)
│   └── Else → action=tool_call (LLM fills arguments as JSON)
│
├── MockExecutor (if tool_call)
│   ├── Validate required params
│   ├── Seed RNG from md5(seed:tool_id:endpoint:sorted_args)
│   ├── Generate deterministic mock field values (name/id/price heuristics)
│   └── Echo _input arguments into output for chain consistency
│
├── memory.add(tool_output, scope="session")  ← enables grounding next step
├── session_state.update(output)              ← enables arg chaining
├── Record ToolCall and ToolOutput
└── step += 1

Post-loop:
├── Calculate memory_grounding_rate
├── ValidatorAgent: rule-based hard checks
├── If valid + corpus enabled: memory.add(summary, scope="corpus")
├── memory.clear_scope("session")
└── Serialize → JSONL line
```

4.3 Validate Phase (`tacs validate`)

- Parse each JSONL line into `Conversation` model.
- Run `ValidatorAgent` on each conversation.
- Print valid/invalid counts.
- Exit code 1 if any invalid conversations found.

4.4 Metrics Phase (`tacs metrics`)

- Single or comparative (Run A vs Run B) mode.
- Reports: avg turns, avg tool calls, avg clarifications, memory grounding rate, multi-step %, multi-tool %, Jaccard diversity of tool sets.
- Comparative mode reports delta (B minus A) for each metric.

---

5. Configuration System

File: `tacs/config.py`

`Config` inherits `pydantic_settings.BaseSettings` :

- `env_file=".env"`
- `env_prefix="TACS_"`
- `extra="ignore"` (unknown env vars silently ignored)

5.1 LLM / Backend Parameters

Env Variable	Default	Description
`TACS_LLM_BACKEND`	`ollama`	`ollama`, `openai`, or `anthropic`
`TACS_LLM_MAX_TOKENS`	`4096`	Max tokens per LLM call
`TACS_OLLAMA_BASE_URL`	`http://localhost:11434`	Ollama server URL

TACS_OLLAMA_MODEL	llama3	Ollama chat model
TACS_OLLAMA_EMBED_MODEL	nomic-embed-text	Ollama embedding model
TACS_OLLAMA_EMBED_DIMS	768	Embedding dimension (must match model)
OPENAI_API_KEY	""	Required for openai backend
TACS_OPENAI_MODEL	gpt-4o-mini	OpenAI chat model
ANTHROPIC_API_KEY	""	Required for anthropic backend
TACS_ANTHROPIC_MODEL	claude-haiku-4-5-20251001	Anthropic chat model

5.2 Generation Parameters

Env Variable	Default	Description
TACS_SEED	42	Random seed for all seeded components
TACS_CONVERSATION_COUNT	50	Number of conversations to generate
TACS_MIN_TOOL_CALLS	3	Hard constraint: minimum tool calls
TACS_MIN_DISTINCT_TOOLS	2	Hard constraint: minimum distinct tools
TACS_MAX_TURNS	10	Maximum turns per conversation
TACS_MAX_RETRIES	50	Sampler retry limit before giving up

5.3 Paths and Operational Settings

Env Variable	Default	Description
TACS_DATA_DIR	data	ToolBench data root
TACS_ARTIFACTS_DIR	artifacts	Build artifacts location
TACS_OUTPUT_DIR	output	JSONL output location
TACS_LOG_LEVEL	INFO	Logging level
TACS_CORPUS_MEMORY_ENABLED	True	Enable corpus memory by default
TACS_MEMORY_TOP_K	5	Top-K results for memory search

5.4 Important Implementation Detail: Config Singleton Safety

A module-level `config = Config()` singleton is created at import time. The `generate` CLI overrides user-supplied seed values using:

```
config = Config().model_copy(update={"seed": seed})
```

Why `model_copy` instead of direct assignment? Direct mutation of the singleton (e.g. `config.seed = seed`) would affect every other component importing `config` — including the sampler and executor — in unexpected ways. `model_copy` creates a fresh isolated instance for that invocation only. See Section 17 for the full reasoning.

---

6. LLM Abstraction

File: `tacs/llm.py`

Class: `LLMClient`

Single interface: `complete(messages: list[dict], **kwargs) -> str`

Supported backends:

Backend	Library	Notes
`ollama`	`ollama.chat(...)`	Requires local Ollama server
`openai`	`openai.chat.completions.create(...)`	Requires `OPENAI_API_KEY`
`anthropic`	`anthropic.messages.create(...)`	System message extracted separately from message list

Behavioral notes:

- No retries, no streaming, no token tracking.
- Returns assistant text as plain string.
- Anthropic requires special handling: the SDK expects the `system` field separately, so the client filters out `role=system` from the message list and passes it as a dedicated parameter.

---

7. CLI Commands

File: `tacs/cli.py`

7.1 `tacs build`

```
tacs build [--data-dir PATH] [--output-dir PATH]
```

- Loads ToolBench tools, builds registry and graph, writes artifacts.
- `--data-dir` does NOT have `exists=True` validation — Click gives a clear error if the path is missing, rather than a cryptic validation failure. The directory creation logic is left to the caller.
- Raises `ClickException` if zero tools are loaded.

Output files:

- `artifacts/registry.pkl`
- `artifacts/graph.pkl`
- `artifacts/build_meta.json` — contains `timestamp`, `tool_count`, `endpoint_count`, `node_count`, `edge_count`

7.2 `tacs generate`

```
tacs generate [--seed N] [--count N] [--output FILE] [--artifacts-dir DIR]
              [--corpus-memory | --no-corpus-memory]
```

- Loads registry and graph from artifacts.
- Runs `ConversationPipeline.generate(...)` writing to JSONL.
- Corpus memory defaults to enabled; use `--no-corpus-memory` for Run A.

7.3 `tacs validate`

```
tacs validate --input FILE
```

- Parses every line as `Conversation`, runs `ValidatorAgent`.
- Prints valid/invalid counts.
- Exits with status code 1 if any invalid conversations are found (useful for CI checks).

7.4 `tacs metrics`

```
tacs metrics [--input FILE]
tacs metrics [--input-a FILE_A] [--input-b FILE_B]
```

Computed metrics:

- Average turns per conversation
- Average tool calls per conversation
- Average clarification questions
- Average memory grounding rate (null values excluded)
- Multi-step rate (conversations with ≥3 tool calls)
- Multi-tool rate (conversations with ≥2 distinct tools)
- Jaccard diversity of tool sets (mean pairwise dissimilarity)
- **Comparative mode**: delta = Run B value − Run A value for each metric

---

8. Registry Subsystem

8.1 Data Models (`tacs/registry/models.py`)

- `Parameter` : `name` , `type` , `required` , `description` , `enum_values` , `default`
- `Endpoint` : `name` , `description` , `method` , `parameters` , `response_fields`
- `Tool` : `tool_id` , `name` , `description` , `category` , `endpoints` , `source_data`

All models use Pydantic with `extra="ignore"` .

8.2 Loader Logic (`tacs/registry/loader.py`)

Primary function: `load_tools(data_dir: Path) -> list[Tool]`

ToolBench JSON format handling (supports format variants):

Raw Field	Normalized To
`tool_name` or `name`	`Tool.name`
`standardized_name`	`Tool.tool_id` (fallback: lowercase name with underscores)
`api_list` or `apis`	`Tool.endpoints`
`required_parameters` or `optional_parameters` = null	Empty list

Enum extraction (`_parse_enum_values`):

- Triggers on: `"one of"` , `"options:"` , `"must be"` in parameter descriptions.
- Extracts quoted values first, then falls back to comma-separated phrase parsing.

Response field loading:

- Reads `data/toolenv/response_examples/<tool_id>/<endpoint>.json` if present.
- Stores top-level JSON keys as `endpoint.response_fields` .

Error resilience:

- Malformed JSON files are logged and skipped — the loader never crashes the pipeline.

8.3 Registry API (`tacs/registry/registry.py`)

Method	Returns	Complexity
`get_tool(tool_id)`	`Tool | None`	O(1)
`get_endpoint(tool_id, endpoint_name)`	`Endpoint | None`	O(n endpoints in tool)
`list_tools()`	`list[Tool]`	O(k tools)
`list_by_category(category)`	`list[Tool]`	O(k)
`list_categories()`	`list[str]`	O(k)
`all_endpoints()`	`list[tuple[str, Endpoint]]`	O(k × e)
`tool_count` / `endpoint_count` / `category_count`	`int`	O(1) or O(k)

Persistence:

- `save(artifacts_dir)` → `registry.pkl` (pickle)
- `load(artifacts_dir)` → raises `FileNotFoundError` if missing

---

9. Graph Subsystem

9.1 Graph Ontology (`tacs/graph/models.py`)

5 Node Types (`NodeType` enum):

Node Type	What It Represents
`TOOL`	An entire API (e.g. `weather_api`)
`ENDPOINT`	A callable action (e.g. `get_forecast`)
`PARAMETER`	An input parameter (e.g. `city`)
`RESPONSE_FIELD`	A field in the API response (e.g. `temperature`)
`CONCEPT`	A semantic category (e.g. `Travel`, `Finance`)

5 Edge Types (`EdgeType` enum):

Edge Type	Direction	Meaning
`HAS_ENDPOINT`	Tool → Endpoint	Tool exposes this endpoint
`HAS_PARAMETER`	Endpoint → Parameter	Endpoint requires/accepts this param
`RETURNS`	Endpoint → ResponseField	Endpoint produces this field
`TAGGED_WITH`	Tool → Concept	Tool belongs to this category
`COMPATIBLE_WITH`	Endpoint → Endpoint	Output of source feeds input of target

Node ID convention (`node_id(node_type, name, parent=None)`):

- `"tool:weather_api"`
- `"endpoint:weather_api.get_forecast"`
- `"parameter:weather_api.get_forecast.city"`

9.2 Graph Builder (`tacs/graph/builder.py`)

Compatibility edge algorithm:

1. Normalize field/parameter names: lowercase, strip underscores/hyphens.
2. Expand aliases via `_ALIASES` dictionary. Examples:
   - `city` ↔ `cityname`, `location`, `place`
   - `id` ↔ `identifier`, `key`
3. Build inverted index: `{normalized_param_name → list[endpoint_id]}`.
4. For each endpoint's response field, look up matching parameter endpoints.
5. Add `COMPATIBLE_WITH` edge from source endpoint to every matching target endpoint.
6. Skip self-loops and duplicate edges.

Why `COMPATIBLE_WITH` matters: This is the backbone of multi-step tool chaining. It means "the output of endpoint A contains a value that endpoint B needs as input." The sampler uses these edges to build realistic API call sequences like `search_flight → book_flight → get_boarding_pass`.

9.3 Tool Chain Sampler (`tacs/graph/sampler.py`)

Model `ToolChain`:

- `steps: list[list[str]]` — parallel groups; most are single-item `[["ep1"], ["ep2"], ["ep3"]]`
- `pattern: str` — `multi_step`, `parallel`, or `hybrid`
- `tool_ids: list[str]` — distinct tools used
- `flat_steps` property — flattens groups: `[["ep1", "ep2"], ["ep3"]] → ["ep1", "ep2", "ep3"]`

Three sampling patterns:

multi_step (Sequential)

1. **Strict path**: Greedily follow `COMPATIBLE_WITH` edges from a random starting endpoint. Build a chain of ≥3 endpoints spanning ≥2 tools.
2. **Loose fallback**: If no COMPATIBLE_WITH chain is long enough, sample endpoints from shared concept nodes. This handles sparse graphs where naming conventions differ too much for alias matching.

parallel (Grouped)

- Pick one concept node.
- Select ≥3 endpoints from ≥2 tools within that concept.
- All endpoints form a single parallel step — they'd be called "simultaneously."

hybrid (Mixed)

- Step 1: One endpoint (preferably one with outgoing compatibility edges).
- Step 2: Its COMPATIBLE_WITH neighbor + one additional concept-compatible endpoint in the same group.
- Result: `[[ep1], [ep2_sequential, ep3_parallel]]`

Sampler guarantees:

- Always produces ≥3 steps and ≥2 distinct tools.
- Retries up to `config.max_retries` (default 50) before failing.
- `SamplerAgent` tries all 3 patterns in random order before giving up.
- Uses local `random.Random(seed)` — isolated from global random state.

---

10. Agent Subsystem

10.1 BaseAgent (`tacs/agents/base.py`)

Abstract class. Stores shared `LLMClient` and `Config`. Requires subclass `run(**kwargs)` implementation.

10.2 Agent Models (`tacs/agents/models.py`)

Model	Key Fields
`Message`	`role` (user/assistant/tool/system), `content`
`ToolCall`	`endpoint`, `arguments`, `step`
`ToolOutput`	`endpoint`, `output` (dict), `step`
`ConversationMetadata`	`seed`, `tool_ids_used`, `num_turns`, `num_clarification_questions`, `memory_grounding_rate`, `corpus_memory_enabled`
`Conversation`	`conversation_id`, `messages`, `tool_calls`, `tool_outputs`, `metadata`
`ConversationPlan`	`scenario`, `domain`, `pattern_type`, `tool_chain`
`AssistantAction`	`action` (clarify/tool_call/respond), `message`, `tool_call`, `grounded`
`ValidationResult`	`valid`, `errors`, `num_tool_calls`, `num_distinct_tools`, `has_clarification`, `memory_grounding_rate`

10.3 SamplerAgent (`tacs/agents/sampler_agent.py`)

- Wraps `ToolChainSampler` with pattern fallback logic.
- Tries patterns in random order until one succeeds.
- Never uses hardcoded tool lists — all chains come from graph traversal.
- No LLM involvement.

10.4 PlannerAgent (`tacs/agents/planner_agent.py`)

Input: `ToolChain` + `corpus_memory_enabled` flag

Corpus memory read (if enabled):

```
query = tool chain as string (tool_ids + endpoints)
results = memory.search(query, scope="corpus", top_k=5)
# injected into LLM prompt as:
# "[Prior conversations in corpus]\n{summaries}\n\n"
```

LLM prompt: System message requires JSON-only output:

```
{"scenario": "...", "domain": "...", "pattern_type": "..."}
```

Fallback (if LLM fails or returns non-JSON):

- Derives domain from keyword matching on tool names (`flight` → `travel` , `stock` → `finance` , `music` → `entertainment`).
- Uses chain pattern name as `pattern_type` .
- Constructs a generic scenario string.

10.5 UserProxyAgent (`tacs/agents/user_proxy.py`)

- **Turn 1**: LLM generates an opening message using the scenario. Intentionally leaves details vague to give the assistant a reason to ask for clarification.
- **Later turns**: LLM responds to the most recent assistant message (answering clarification questions).
- **Fallback**: Generic message derived from scenario string on LLM failure.

10.6 AssistantAgent (`tacs/agents/assistant_agent.py`)

This is the most complex agent. It runs in phases:

Phase 0: If `final=True` → generate closing response via LLM.

Phase 1 (non-first steps only): Read session memory:

```
results = memory.search(current_endpoint_name, scope="session", top_k=5)
grounded = len(results) > 0
```

Phase 2: Rule-based action decision:

```
IF step == 0
   AND required_params exist for this endpoint
   AND no prior assistant message in history
THEN action = "clarify"
ELSE action = "tool_call"
```

Why rule-based and not LLM-decided? The LLM can hallucinate or be inconsistent about when to ask clarification. A deterministic rule ensures the conversation always follows the same structural pattern, making the dataset easier to analyze and the pipeline easier to debug. The LLM is only used to *write* the clarification question text, not to decide *whether* to ask. See Section 17.

Phase 3a (clarify): LLM generates a clarifying question asking for the missing required parameters.

Phase 3b (tool_call): LLM generates arguments as JSON. Then `session_state` values (exact outputs from prior steps) are merged in, ensuring chained arguments are always correct even if the LLM's JSON is imperfect.

Fallback:

- Endpoint resolution failure at step 0 → generic clarify.
- Endpoint resolution failure at later steps → use current session_state as arguments.

10.7 ValidatorAgent (`tacs/agents/validator_agent.py`)

Pure rule-based. No LLM. Validates a completed `Conversation` object.

Hard constraints (failure = invalid conversation):

Check	Threshold
Number of tool calls	≥ `config.min_tool_calls` (default 3)
Number of distinct tools	≥ `config.min_distinct_tools` (default 2)
Messages list non-empty	True
Tool call steps sequential	`[0, 1, 2, ...]`
`memory_grounding_rate` in range	`[0.0, 1.0]` or null
`num_turns > 0`	True
`tool_ids_used` non-empty	True
`num_clarification_questions >= 0`	True

Tracked but NOT a hard constraint:

- `has_clarification` : Whether any assistant message contains `?` . Recorded in `ValidationResult` for analysis but a conversation with zero clarification questions still passes validation.

Important: The validator does NOT check that `len(tool_outputs) == len(tool_calls)` . That alignment is structurally guaranteed by the pipeline (MockExecutor always produces one output per call), so checking it in the validator

would be redundant.

10.8 Pipeline Orchestration (`tacs/agents/pipeline.py`)

Class `ConversationPipeline.run(seed, corpus_memory_enabled)` :

1. Sample chain → plan → turn loop → compute grounding rate → validate → write corpus memory → clear session memory.
2. Wraps the entire run in a try/except — unexpected exceptions log the traceback but still return whatever `Conversation` was partially built.
3. Always clears session memory in a finally block (even on exception).

Batch function `generate(count, output_path, ...)` :

- Creates output directory.
- Writes each conversation immediately (one line at a time) — no buffering.
- Uses `tqdm` progress bar.
- Conversation IDs: `conv_00000` , `conv_00001` , ...

---

11. Execution Subsystem

File: `tacs/execution/executor.py`

MockExecutor

`execute(tool_id, endpoint_name, arguments) -> MockResult`

Step 1: Validation

- Checks endpoint exists in registry.
- Checks all required parameters are present in arguments.

Step 2: Deterministic RNG seeding

```
hash_input = f"{seed}:{tool_id}:{endpoint_name}:{sorted_args_json}"
hash_hex = md5(hash_input.encode()).hexdigest()
rng = random.Random(int(hash_hex, 16))
```

This ensures: same tool call + same seed = same output on any machine, any OS.

Step 3: Field value generation (heuristics by field name)

Field name contains	Generated value
`id`	`"endpoint_name_123"`
`name`	Realistic person/place name
`price` or `cost`	Integer (e.g. 850)
`date`	Date string (e.g. `"2024-03-18"`)
`url`	Fake URL
`status`	One of `active`, `pending`, `confirmed`
`email`	Fake email address
Other	Random string or number

Step 4: Echo inputs

- The output dict always includes `_input: arguments` — so downstream steps can access the original input that produced this output.

Why mock execution? Real APIs would be slow, costly, rate-limited, and non-deterministic. For synthetic data generation the goal is structural correctness, not API accuracy. See Section 17.

---

12. Memory Subsystem

File: `tacs/memory/store.py`

Architecture

- **Backend**: `mem0` library
- **Vector store**: Qdrant configured with `path=":memory:"` (in-memory, not file-based)
- **LLM for extraction**: Backend-dependent
- **Embedder**: Backend-dependent (see table below)

Backend Configuration

`TACS_LLM_BACKEND`	LLM Used	Embedder Used	Embedding Dims
`ollama`	Ollama (`llama3`)	Ollama (`nomic-embed-text`)	768
`openai`	OpenAI (`gpt-4o-mini`)	OpenAI (`text-embedding-3-small`)	1536
`anthropic`	Anthropic (`claude-haiku-...`)	OpenAI (`text-embedding-3-small`)	1536

Anthropic special case: Anthropic has no embedding API. When `TACS_LLM_BACKEND=anthropic`, `OPENAI_API_KEY` is **also required** for the embedder. The store raises a `ValueError` at initialization if the key is missing.

Scope Isolation

Scopes are implemented via mem0's native `user_id` parameter:

- Session scope: `user_id="session"`
- Corpus scope: `user_id="corpus"`

Cross-scope leakage is impossible — mem0 treats each `user_id` as a completely separate namespace.

Why `path=":memory:"` for Qdrant?

This is a deliberate fix for a macOS threading bug. The full explanation:

File-based Qdrant on macOS uses SQLite internally. SQLite's default configuration sets `check_same_thread=True`, which raises an error when a connection is accessed from a thread other than the one that created it. When mem0 performs vector operations, it uses background threads that call back into Qdrant's SQLite connection — triggering this error.

Using `path=":memory:"` bypasses the file-based SQLite layer entirely and avoids the threading issue. The side effect is that memory is non-persistent across runs, which is acceptable (and actually desirable) for this pipeline. See Section 17.

Why `infer=False` in `memory.add()`?

By default, mem0 runs an LLM extraction step when storing a memory: it takes raw input and rewrites it as a condensed semantic "fact." For example, raw tool output `{"flight_id": "F99", "price": 850}` might become "The user found a flight F99 for $850."

TACS disables this with `infer=False` because:

1. **We want the raw data**, not a summarized version. The assistant needs exact values (like `flight_id`) to chain into the next tool call.
2. **It's faster** — skips one LLM call per tool step.
3. **It's more reproducible** — no LLM summarization variability.

Note: `infer=False` skips the *LLM extraction* step only. Embeddings are still computed and stored so that `memory.search()` can do semantic similarity lookups.

Memory Operations Summary

Operation	When	Scope	What
`memory.add(tool_output)`	After each tool execution	session	Raw JSON dump of output
`memory.search(endpoint_name)`	Before filling non-first tool args	session	Retrieve prior outputs
`memory.clear_scope("session")`	End of each conversation	session	Reset for next conversation
`memory.add(summary)`	After valid conversation (if corpus enabled)	corpus	Compact summary string
`memory.search(chain_str)`	In PlannerAgent (if corpus enabled)	corpus	Prior conversation context

Memory Grounding Rate

```
rate = (# non-first tool calls where memory.search() returned ≥1 result)
       ÷ (# non-first tool calls)
```

- `null` if there are no non-first tool calls (only one tool call total).
- A call is "grounded" if `len(search_results) > 0` — no score threshold is applied.
- In practice, this rate is very often 1.0 because the immediately preceding tool output was just written to session memory and will almost always be retrieved. This limits the metric's discriminative value — see Section 18.

---

13. Data Contracts and JSON Output

Each generated JSONL line serializes a `Conversation` object:

```
{
  "conversation_id": "conv_00042",
  "messages": [
    {"role": "user", "content": "I need help booking a trip to Tokyo."},
    {"role": "assistant", "content": "Could you tell me your departure city and travel dates?"},
    {"role": "user", "content": "Flying from NYC, leaving March 20, returning March 27."},
    {"role": "assistant", "content": "Let me search for available flights."},
    {"role": "tool", "content": "{\"flight_id\": \"F99\", \"price\": 850, \"_input\": {...}}"},
    {"role": "assistant", "content": "I found a flight for $850. Let me book it."},
    {"role": "tool", "content": "{\"booking_id\": \"B123\", \"confirmation_code\": \"ABCD\",
\"_input\": {...}}"},
    {"role": "assistant", "content": "Your flight is booked! Confirmation: ABCD."}
  ],
  "tool_calls": [
    {"endpoint": "flight_search", "arguments": {"origin": "NYC", "destination": "Tokyo", "date":
"2024-03-20"}, "step": 0},
    {"endpoint": "book_flight", "arguments": {"flight_id": "F99", "cabin_class": "economy"},
"step": 1}
```

```
  ],
  "tool_outputs": [
    {"endpoint": "flight_search", "output": {"flight_id": "F99", "price": 850, "_input": {...}},
"step": 0},
    {"endpoint": "book_flight", "output": {"booking_id": "B123", "confirmation_code": "ABCD",
"_input": {...}}, "step": 1}
  ],
  "metadata": {
    "seed": 42,
    "tool_ids_used": ["flight_api", "booking_api"],
    "num_turns": 8,
    "num_clarification_questions": 1,
    "memory_grounding_rate": 1.0,
    "corpus_memory_enabled": true
  }
}
```

Key semantic notes:

- `num_turns` = total number of `messages` (user + assistant + tool).
- `tool_ids_used` = distinct tools from sampled chain (ordered).
- `memory_grounding_rate = null` when only one tool call exists.
- `_input` in tool output = echo of the exact arguments passed in.

Build artifacts (`artifacts/build_meta.json`):

```
{
  "timestamp": "2024-03-18T14:30:00+00:00",
  "tool_count": 47,
  "endpoint_count": 312,
  "node_count": 2145,
  "edge_count": 8392
}
```

---

14. Testing Strategy

14.1 Unit Tests: Registry (`tests/unit/test_registry.py`)

- Enum extraction parser for all trigger phrases.
- Parameter parsing: required/optional, default type coercion.
- Endpoint parsing: method normalization, response fields.
- Loader: valid directories, malformed JSON (skipped, not crashed), missing directories.
- Registry API: all lookup methods, aggregate counts, pickle save/load lifecycle.

14.2 Unit Tests: Graph and Sampler (`tests/unit/test_graph.py`)

- All 5 node types present in built graph.
- All 5 edge types present.
- `COMPATIBLE_WITH` edges exist in synthetic fixture.
- `ToolChain.flat_steps` flattening for sequential/parallel/hybrid.
- Sampler produces valid chains for all three patterns.
- Constraints enforced: `min_steps` , ≥2 distinct tools.
- Error conditions: invalid pattern name, invalid `min_steps` .
- Determinism: same seed → same chain.

14.3 Unit Tests: Memory (`tests/unit/test_memory.py`)

- `add` → `search` round-trip returns results.
- Result shape and content.
- `top_k` limits respected.
- Scope isolation: session results don't appear in corpus searches and vice versa.
- `clear_scope` : clears target scope without affecting other scope.
- **Conditional skip**: entire module is skipped (with an explanation message) if `MemoryStore()` cannot initialize — e.g. Ollama not running or API key absent. Tests do not hard-fail on missing infrastructure.

14.4 End-to-End Tests (`tests/e2e/test_pipeline.py`)

Preconditions (skip if not met):

- `data/` directory must exist.
- Artifacts and output JSONL files must be pre-generated.

Coverage:

- Artifact files exist and are loadable.
- `build_meta.json` has all required fields.
- Graph has ≥4 node types.
- Run A and Run B each have ≥50 conversations.
- All conversations have required fields.
- `corpus_memory_enabled` flag matches expected value per run.
- Multi-step conversations present (≥3 calls).
- Multi-tool conversations present (≥2 tools).
- `len(tool_outputs) == len(tool_calls)` for all conversations.
- Conversation IDs are unique within each run.
- `memory_grounding_rate` in `[0, 1]` where not null.

Note: E2E tests do not invoke CLI commands. They validate pre-generated artifacts and datasets.

---

15. Determinism and Reproducibility

What is deterministic

Component	How
Tool chain sampling	`random.Random(seed)` local RNG
Mock executor outputs	MD5 hash of `(seed, tool_id, endpoint, sorted_args)` seeds RNG
Conversation IDs	Sequential counters `conv_00000`, ...

What is NOT deterministic

Component	Why
LLM-generated text	Model temperature, sampling, API version
Memory retrieval ranking	Embedding model + vector index internals

Practical implication

The **structure** of the dataset (which tools are called, in which order, with what mock outputs) is stable across runs with the same seed. The **natural language** (scenario text, user messages, assistant messages) may vary by LLM backend or version. For research purposes, the structural trace is what matters for tool-use training.

---

16. Error Handling Philosophy

Principle: prefer graceful degradation over hard failure during generation.

Situation	Behavior
Malformed ToolBench JSON	Log and skip file; continue loading
Memory `add` failure	Log warning; continue pipeline
Memory `search` failure	Return empty list; call treated as ungrounded
LLM parse failure (non-JSON)	Fall back to deterministic default plan/message/args
Unexpected pipeline exception	Log traceback; clear session memory; return partial Conversation
Sampler exhausts retries	SamplerAgent tries next pattern
All patterns fail	Exception bubbles to pipeline error handler

Hard failures (appropriate):

- Missing build artifacts → `FileNotFoundError`
- Zero tools found during build → `ClickException`
- `validate` command finds invalid conversations → exit code 1
- `anthropic` backend + missing `OPENAI_API_KEY` → `ValueError` at startup

---

17. Design Decisions and Rationale

This section explains the **why** behind implementation choices that would not be obvious from reading the code alone.

Decision 1: `path=":memory:"` for Qdrant

Problem: On macOS, file-based Qdrant uses SQLite with `check_same_thread=True`. When mem0 performs vector operations using background threads, those threads call back into the same SQLite connection that was opened on the main thread — which raises a `ProgrammingError`.

Solution: `path=":memory:"` uses an in-memory Qdrant store that bypasses the file-based SQLite layer entirely, eliminating the threading issue.

Side effects: Memory is non-persistent across runs. This is acceptable — session memory is cleared after each conversation anyway, and corpus memory is rebuilt each generation run.

Alternative considered: File-based Qdrant with `check_same_thread=False` patching — rejected because it requires patching a library internal, which is fragile on version changes.

Decision 2: `infer=False` in `memory.add()`

Problem: mem0's default behavior runs an LLM extraction pass on every `add()` call, summarizing raw content into a semantic "memory fact." For tool outputs like `{"flight_id": "F99", "price": 850}`, this would produce something like "The user found flight F99 for $850" — losing the exact `flight_id` value needed by the next tool call.

Solution: `infer=False` stores the content verbatim, without LLM extraction. Embeddings are still computed (at write time) so `search()` can do semantic retrieval.

Why this matters for chaining: The AssistantAgent merges exact `session_state` values into tool arguments. If the stored memory had been paraphrased by LLM extraction, that exact value retrieval would break.

Decision 3: `Config().model_copy(update={"seed": seed})`

Problem: `config` is a module-level singleton. If the CLI mutated it directly (`config.seed = user_seed`), every other module importing `config` would see the mutated value — including future calls in the same process if the pipeline were ever run in a loop or test.

Solution: `model_copy(update={...})` creates a fresh `Config` instance with the override applied, leaving the singleton untouched.

Why this matters in tests: Unit tests may import the same `config` singleton. Mutation would cause tests to interfere with each other.

Decision 4: Rule-based clarification decision

Problem: Whether to ask a clarifying question could be decided by an LLM, but LLMs are inconsistent — sometimes they clarify when they shouldn't, sometimes they skip clarification and hallucinate arguments.

Solution: The clarify/tool_call decision is purely rule-based:

```
clarify IF (step==0 AND required_params_missing AND no_prior_assistant_msg)
```

The LLM is only used to *write* the clarification question text.

Benefits:

1. Deterministic: every run produces the same structural decision for the same input.
2. Debuggable: the rule is explicit and auditable.
3. Consistent dataset: clarification patterns are uniform, making them easier to analyze.

Decision 5: Pickle for artifacts

Problem: Need to persist `ToolRegistry` (a Python object) and a NetworkX `DiGraph` between the build and generate phases.

Solution: Python `pickle` — simple, zero-dependency, handles arbitrary Python objects including NetworkX graphs.

Trade-off: Pickle files are Python-version and class-structure-coupled. If `ToolRegistry` changes its fields, old artifacts break. For an offline research pipeline this is acceptable. A JSON/Parquet export would be more portable but requires serialization logic for custom types.

Decision 6: Loose sampler fallback

Problem: `COMPATIBLE_WITH` edges depend on response field names matching required parameter names (with alias expansion). In sparse ToolBench data, many endpoints have no naming overlap, so the strict COMPATIBLE_WITH chain may not reach ≥3 steps.

Solution: The loose fallback samples endpoints that share a concept node (category), even without COMPATIBLE_WITH edges. This treats "same category = likely usable together" as a weaker compatibility signal.

Why this is necessary: Without it, the sampler would fail on most tool combinations in ToolBench, causing the pipeline to stall. The loose fallback ensures the pipeline always produces output.

Decision 7: Mock execution instead of real API calls

Rationale:

1. Real APIs require credentials, have rate limits, cost money, and are slow.
2. Real APIs introduce non-determinism — responses change over time.

3. For training data, structural correctness (correct tool selected, correct arguments shaped) matters more than realistic response content.
4. The `_input` echo in mock output is actually more useful than a real response for chain verification, since it proves the arguments were passed correctly.

Decision 8: Session state merging for argument chaining

Problem: The LLM fills tool arguments as JSON. But it may not correctly reproduce exact values from prior steps (e.g. `flight_id: "F99"` from step 0 into `book_flight` at step 1).

Solution: After LLM fills arguments, the pipeline merges exact `session_state` values (accumulated from all prior tool outputs) on top. This guarantees chained arguments are always correct, regardless of LLM accuracy on the JSON fill.

---

18. Known Constraints and Trade-offs

1. Memory grounding rate is trivially 1.0 in practice

Every non-first tool call reads session memory. Since the immediately prior tool output was just written to session memory, `search()` almost always returns at least one result. This makes `memory_grounding_rate` close to 1.0 for almost all conversations. As a quality metric it has limited discriminative power — it measures whether memory *retrieval happened*, not whether memory *influenced the arguments*.

Implication: A richer grounding metric would measure whether retrieved values actually appeared in the arguments, not just whether any result was returned.

2. Pickle artifacts are not portable

`registry.pkl` and `graph.pkl` are coupled to Python version and class structure. Changing `ToolRegistry` fields invalidates existing artifacts. For long-lived pipelines, a language-neutral format (JSON, Parquet) would be safer.

3. Clarification detection uses question-mark heuristic

The validator counts clarification questions by checking for `?` in assistant messages. This can over-count (rhetorical questions, uncertain phrasing) or under-count (clarifications phrased as statements).

4. Memory operation failures are silent

`memory.add()` and `memory.search()` both catch exceptions and return silently (empty list or no-op). This prevents pipeline crashes but means memory degradation is invisible without monitoring logs.

5. E2E tests require pre-generated artifacts

`tests/e2e/test_pipeline.py` validates existing artifacts and JSONL files — it does not generate them. In CI, this requires either committing the artifacts (bad for large files) or a setup step that runs `tacs build` + `tacs generate` before tests.

6. LLM prompt contracts are fragile

All LLM prompts expect strict JSON output. The code strips fenced code blocks (```` ```json ... ``` ````) and attempts JSON parsing with a fallback, but sufficiently malformed LLM output still degrades to the fallback plan/message. This is acceptable for generation but means dataset quality depends on the LLM's instruction-following ability.

7. COMPATIBLE_WITH edges rely on naming conventions

Compatibility matching uses name aliases but is ultimately heuristic. Two semantically compatible endpoints with unconventionally named fields/parameters won't be linked by a `COMPATIBLE_WITH` edge, degrading multi-step chain quality for those tool pairs.

8. Mock outputs don't reflect real API behaviour

Tool outputs are synthetically generated from field-name heuristics. The dataset teaches tool *selection* and *argument structure*, but not how to interpret real API responses — error codes, pagination, partial failures, etc. are absent.

9. LLM argument filling can hallucinate values

Session state merging only fixes values that appear verbatim in prior tool outputs. Novel required parameters (those not produced by any prior step) are filled entirely by the LLM and may be plausible but semantically wrong (e.g. a date in the past, an invalid location name).

10. Evaluating realism of generated conversations

TACS does not have a built-in realism evaluator. The following approaches can be used externally:

- **Human evaluation**: Sample conversations and have human raters score naturalness, coherence, and tool-call correctness.
- **Downstream model performance**: Train a tool-use model on TACS data and measure its performance on real benchmarks. Better training data should produce higher benchmark scores.
- **Distribution comparison**: Compare TACS dataset statistics (turn length, vocabulary, tool-call patterns, argument value distributions) against human-annotated tool-use datasets such as ToolBench's own human demonstrations.
- **Argument correctness audit**: Check what fraction of LLM-generated arguments match the expected type and value range for each parameter as declared in the tool schema.

11. Scalability considerations

The current pipeline is single-process and sequential. Bottlenecks at larger scale:

Bottleneck	Impact	Mitigation
Sequential conversation generation	Linear time	Parallelize with `multiprocessing` or async workers
In-memory Qdrant corpus	Grows unbounded with conversation count	Migrate to file-based or hosted Qdrant at scale
Single JSONL output file	Write contention at high concurrency	Partition by seed range; merge after
LLM calls per turn	Slowest step in each turn	Batch where possible; use faster/cheaper model for high volume
E2E tests need pre-generated artifacts	CI setup complexity	Add a CI setup step running `tacs build` + `tacs generate` on fixture data

---

19. Extension Points

1. Real tool execution

Replace `MockExecutor` with a `LiveExecutor` that calls real APIs in a sandboxed environment. The interface (`execute(tool_id, endpoint_name, arguments) -> MockResult`) is unchanged — only the implementation differs.

2. Memory-aware sampling

Feed corpus memory into `ToolChainSampler` to preferentially sample tool combinations not yet well-represented in the corpus. This would directly optimize dataset diversity rather than relying on the planner's indirect influence.

3. LLM retry and backoff in `LLMClient`

Add exponential backoff for transient failures (rate limits, timeouts). Currently there is no retry logic — a transient failure falls through to the fallback immediately.

4. Richer validation

Add semantic checks: does the conversation scenario match the tools used? Are the argument values realistic (e.g. dates in the future, valid location names)?

5. Non-pickle artifact formats

Export `ToolRegistry` as JSON and the graph as adjacency lists for inspection and cross-language use.

6. Pipeline integration tests

Invoke CLI commands end-to-end in temporary fixtures (temp directories, fixture ToolBench data) to test the full `build → generate → validate → metrics` flow without pre-generated artifacts.

7. Better grounding metric

Track whether memory retrieval results actually appeared in tool arguments (not just whether results were returned), giving a more meaningful signal for whether corpus memory improved argument quality.

---

20. Packaging and Dependencies

`pyproject.toml`

- Project name: `tacs`, version `0.1.0`
- Python requirement: `>=3.11`
- Console script: `tacs = tacs.cli:cli`
- Tools: Black (line length 88), isort (profile black), pytest (testpaths: `tests`)

Runtime Dependencies

Package	Role
`pydantic` + `pydantic-settings`	Data models and env-backed config
`networkx`	Tool compatibility graph
`click`	CLI framework
`mem0ai`	Vector memory (session + corpus)
`openai`	OpenAI backend + embeddings
`anthropic`	Anthropic backend
`ollama`	Ollama backend
`python-dotenv`	`.env` file loading
`faker`	Realistic mock values in executor
`tqdm`	Progress bar during generation
`qdrant-client`	Vector store backend for mem0

`requirements.txt` contains fully pinned versions (`pip freeze` output) for exact reproducibility.

---

21. Operational Runbook

Initial Setup

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
pip install -e .
cp .env.example .env
# Edit .env to set TACS_LLM_BACKEND and relevant API keys
```

Build Artifacts

```
tacs build --data-dir data --output-dir artifacts
```

Expected output:

- `artifacts/registry.pkl`
- `artifacts/graph.pkl`
- `artifacts/build_meta.json`

Generate Datasets

Run A (no corpus memory — baseline):

```
tacs generate --seed 42 --count 50 --no-corpus-memory --output output/run_a.jsonl
```

Run B (corpus memory enabled — experimental):

```
tacs generate --seed 42 --count 50 --output output/run_b.jsonl
```

Validate and Compare

```
tacs validate --input output/run_b.jsonl
tacs metrics --input output/run_b.jsonl
tacs metrics --input-a output/run_a.jsonl --input-b output/run_b.jsonl
```

Run Tests

```
pytest tests -v
```

Unit tests run without any setup. Memory tests skip if the backend is unavailable. E2E tests skip if `data/` is absent.

---

22. Security and Data Considerations

- Generated outputs are fully synthetic. No real user data is involved.
- Mock execution does not call real APIs by default.
- API keys (`OPENAI_API_KEY` , `ANTHROPIC_API_KEY`) should be stored in `.env` only, never committed to source control.
- `.env` is listed in `.gitignore` .

- Pickled artifacts (`registry.pkl` , `graph.pkl`) should be treated as trusted inputs only — pickle deserialization executes arbitrary Python code. Do not load artifacts from untrusted sources.

---

23. File-by-File Index

Top-level

- `.env.example` — Environment variable template
- `README.md` — User-facing setup and run guide
- `DESIGN.md` — Architecture narrative and experiment framing
- `pyproject.toml` — Package config, CLI entrypoint, tooling settings
- `requirements.txt` — Pinned dependency lock

`tacs/`

- `config.py` — Pydantic settings model + `config` singleton
- `llm.py` — `LLMClient` with ollama/openai/anthropic backends
- `cli.py` — Click group with build/generate/validate/metrics commands

`tacs/registry/`

- `models.py` — `Tool` , `Endpoint` , `Parameter` Pydantic models
- `loader.py` — `load_tools()` : ToolBench JSON parser and normalizer
- `registry.py` — `ToolRegistry` : lookup index + pickle persistence

`tacs/graph/`

- `models.py` — `NodeType` , `EdgeType` enums + `node_id()` helper
- `builder.py` — `ToolGraphBuilder` : graph construction + compatibility edges
- `sampler.py` — `ToolChainSampler` + `ToolChain` model

`tacs/agents/`

- `base.py` — `BaseAgent` abstract class
- `models.py` — All conversation Pydantic models
- `sampler_agent.py` — `SamplerAgent` : chain sampling with fallback
- `planner_agent.py` — `PlannerAgent` : scenario planning with corpus memory
- `user_proxy.py` — `UserProxyAgent` : user utterance simulation
- `assistant_agent.py` — `AssistantAgent` : decision + argument filling
- `validator_agent.py` — `ValidatorAgent` : rule-based conversation validation
- `pipeline.py` — `ConversationPipeline` : full orchestration + JSONL writer

`tacs/execution/`

- `executor.py` — `MockExecutor` + `MockResult`

`tacs/memory/`

- `store.py` — `MemoryStore` : mem0 wrapper with session/corpus scope isolation

`tests/`

- `unit/test_registry.py` — Registry parser/loader/lookup tests
- `unit/test_graph.py` — Graph construction and sampler tests
- `unit/test_memory.py` — Memory add/search/isolation/clear tests (conditional skip)
- `e2e/test_pipeline.py` — Artifact and dataset-level output validation

---

24. Quick Reference Commands

```
# Setup
pip install -r requirements.txt && pip install -e .

# Build
tacs build --data-dir data --output-dir artifacts

# Generate
tacs generate --seed 42 --count 50 --no-corpus-memory --output output/run_a.jsonl
tacs generate --seed 42 --count 50 --output output/run_b.jsonl

# Validate
tacs validate --input output/run_b.jsonl

# Metrics — single run
tacs metrics --input output/run_b.jsonl

# Metrics — A/B comparison
tacs metrics --input-a output/run_a.jsonl --input-b output/run_b.jsonl

# Tests
pytest tests -v
pytest tests/unit -v          # unit tests only (no artifacts needed)
pytest tests/e2e -v           # e2e tests (requires pre-built artifacts)
```